

Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction Using Machine Learning
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Developer Tools
Type of Testing	Load Testing, Response Time Testing
Target Module / API	Credit Card Approval Prediction Module (/predict)
Test Environment	Local Machine (Flask Server)
Test Date	03-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit a valid applicant record for prediction	1	30	Prediction displayed successfully within 2 seconds
2	Send multiple prediction requests simultaneously	20	60	System remains responsive with no failures
3	Submit invalid or incomplete applicant data	10	30	Appropriate validation error messages displayed
4	Continuous prediction requests	30	120	Stable performance without application crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.85 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.10 seconds	Pass	Stable under continuous requests
3	Throughput (Req/sec)	> 10 Req/sec	15 Req/sec	Pass	Good request handling capability
4	Error Rate	< 1%	0%	Pass	No failed prediction requests
5	CPU Utilization	< 80%	6%	Pass	CPU usage remained low during execution.
6	Memory Utilization	< 80%	58%	Pass	Memory usage remained within acceptable limits.

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction system successfully processed all prediction requests with an **HTTP Status Code 200 (OK)**, indicating successful communication between the client and the Flask server.
- The average response time was approximately **0.85 seconds**, which is well below the target threshold of **2 seconds**, ensuring a fast user experience.
- CPU utilization remained low at **6%**, demonstrating efficient processing during prediction requests.
- Memory utilization was **58%**, remaining well within the acceptable limit of **80%**, indicating stable memory usage.
- No failed requests or application crashes were observed during testing, resulting in an **error rate of 0%**.

Bottlenecks Identified

- No significant performance bottlenecks were identified during testing under the expected workload.
- Minor increases in response time may occur when multiple prediction requests are submitted simultaneously due to local system resource sharing and Flask's development server limitations.
- As the application currently runs on a local development environment, performance may vary depending on the hardware and system load.

Optimization Steps Taken

- The trained machine learning model was saved and loaded using **Joblib**, eliminating the need to retrain the model for every prediction request.
- Data preprocessing was optimized to process only the required applicant information before prediction, reducing unnecessary computations.
- Input validation was implemented to reject invalid or incomplete data before reaching the prediction module, improving reliability and reducing processing overhead.
- The application follows a modular architecture, separating the frontend, backend, and machine learning components, which improves maintainability and execution efficiency.
- Efficient Python libraries such as **Pandas**, **NumPy**, and **Scikit-learn** were used to ensure fast data processing and prediction.

Step 5: Screenshots / Evidence



